

T9 in Java

Mostafa Faik

October 20, 2024

Introduction

In this assignment, we implemented a basic version of the T9 predictive text input system, which was commonly used on mobile phones with traditional numeric keypads. The T9 system helps users efficiently compose text messages by predicting words based on the numeric keys pressed. This report details the implementation of the T9 system using a trie data structure, specifically designed to store and retrieve common Swedish words. We explore the structure of the trie, the encoding and decoding of key sequences, and the handling of special Swedish characters.

A Trie Data Structure

The trie is a tree-like data structure that efficiently stores words by representing each character as a node. Each node has branches corresponding to the characters in the Swedish alphabet ('a' to 'ö'), and paths from the root node to a leaf node represent complete words. In our implementation, each node can have up to 27 branches corresponding to the Swedish alphabet characters (a-ö), excluding the letters 'q' and 'w'. This design allows us to efficiently store and retrieve words by traversing the trie according to the characters in each word.

Node Structure

The `Node` class in our implementation represents a single character in the trie and contains an array of references to its child nodes and a boolean flag indicating if it marks the end of a valid word.

```
private class Node {  
    public Node[] next; // Array to store the next nodes  
                        // (each representing a letter)  
    public boolean valid; // Marks if this node  
                        // represents a valid word  
}
```

```

    public Node() {
        next = new Node[27]; // One branch for each
                               // Swedish letter (a to ö, no q or w)
        valid = false;
    }
}

```

Each **node** has 27 branches, corresponding to each possible **Swedish letter**, excluding 'q' and 'w.' A **node** with no further branches represents a **leaf** in the **trie** and indicates the end of a valid word. The valid **boolean** flag signifies if the path from the **root** to that **node** forms a complete word from the list.

Why Use a Trie?

The **trie** structure allows us to efficiently store and search for words. When you type a **sequence** of numbers, the system can easily follow **branches** corresponding to the **characters** represented by those **numbers**, reducing the search space. Although the **trie** can be space-consuming due to the 27 **branches**, it is still a good solution for fast lookups.

Encoding Words into the Trie

The first task was to load the list of **Swedish words** from the provided file (**kelly.txt**) into a **trie**. Each word was added character by character into the **trie**, where each character corresponds to a **node**. When we reach the last character of a word, we mark the respective **node** as a valid word. This allowed us to efficiently store the entire word list and perform **prefix-based** lookups.

Character Encoding and Mapping

Since T9 maps groups of letters to specific numeric keys (e.g., 2 → a, b, c; 3 → d, e, f), we needed to create a method that converts characters into numeric values. This was done by creating a mapping between the characters and their corresponding T9 **keys**.

```

private int charToKey(char c) {
    // Mapping Swedish letters to numeric keys
    if (c == 'a' || c == 'b' || c == 'c') return 2;
    // Continue for all relevant characters...
}

```

The *charToKey()* method returns the numeric key for a given letter.

Word Lookup

To retrieve possible words that match a given sequence of T9 **keys**, we implemented a **lookup** procedure that starts at the **root** of the **trie** and follows the branches corresponding to each **digit** in the key sequence. For example, given the sequence "43," the system would search for words starting with "g," "h," or "i" (from key 4) followed by "d," "e," or "f" (from key 3).

While this basic implementation works, the original T9 system was much more memory-efficient. The **trie** structure can be optimized in several ways, including compressing **nodes** that represent single branches or using more memory-efficient data types.

Code, Index, and Key

In this part of the assignment, we were tasked with implementing several methods to handle the **character encoding**, **key mapping**, and **reverse mappings**. These methods are essential for efficiently addressing the branches of our **trie** structure and for **encoding** and **decoding** words based on T9 key sequences.

Character to Code Conversion

We first needed to convert **characters** into **integer codes**. This is necessary for addressing the branches in the **trie** since each **node** has an array of branches representing the 27 Swedish letters ('a' to 'ö', excluding 'q' and 'w'). The index of the branch in the array will correspond to the code of the character.

- Character (a..ö): Convert each character to a code between 0 and 26.
- Code (0..26): This will be used to access the respective branch in a **node's** *next[]* array.

```
private static int code(char w) {
    switch (w) {
        case 'a': return 0;
        case 'b': return 1;
        .
        .
        case 'ä': return 25;
        case 'ö': return 26;
        default: return -1;
    }
}
```

Code to Character Conversion

Next, we needed the reverse of the *code()* method: converting an **integer** code back into its respective character. This is useful for cases where we want to **decode** a **key** sequence back into a **string** of characters.

```
private static char character(int code) {
    switch (code) {
        case 0: return 'a';
        case 1: return 'b';
        .
        .
        case 25: return 'ä';
        case 26: return 'ö';
        default: return '?';
    }
}
```

Key to Index Conversion

The T9 system uses **keys** like '1', '2', etc. These keys map to groups of characters. For example, key '2' corresponds to the letters 'a', 'b', and 'c'. We need a method that converts a key ('1', '2', etc.) into an **index** (0..8), which will be used to traverse the trie.

```
private static int keyToIndex(char key) {
    switch (key) {
        case '1': return 0;
        case '2': return 1;
        .
        .
        case '9': return 8;
        default: return -1; // Invalid key
    }
}
```

Character to Key Mapping

This method maps each character to the respective T9 **key**. This is useful when **encoding** a word into its corresponding **numeric sequence**. For example, the word "toffel" should be encoded as "752224".

```
private static char charToKey(char c) {
    switch (c) {
        case 'a': case 'b': case 'c': return '2';
```

```

        .
        .
        case 'å': case 'ä': case 'ö': return '9';
        default: return '?'; // Invalid character
    }
}

```

Adding Words to the Trie

The next step in the assignment was to populate the **trie** structure with words. The process of **adding words** was straightforward: for each word in the **list**, we traverse the **trie** from the **root node**, moving down one level for each character in the **word**. If a branch doesn't exist for a particular character, we create a new **node**. Once we reach the last character, we mark the **node** as a valid **node** to indicate that the path from the **root** to this **node** forms a complete **word**. The approach was as follows:

- **Start at the Root:** For each **word**, begin at the **root** of the **trie**.
- **Traverse Down:** For each character in the **word**, find the corresponding **index** (using the *code()* method we implemented earlier) and move to the corresponding branch in the *next[]* array.
- **Create Nodes as Needed:** If a **node** for the current character doesn't exist, create a new **Node** at that branch.
- **Mark the End:** Once we reach the last character of the **word**, mark the **node** as valid by setting its valid flag to **true**.

Searching for Words Given a Key Sequence

In this part of the assignment, we implemented a procedure called **decode** that takes a T9 key sequence (e.g., "2314") and returns a **list** of all possible **words** that could match the sequence. The goal was to traverse the **trie**, exploring all branches that correspond to the **key** sequence and collecting valid **words** along the way. We approached this as follows:

- **Starting Point:** We begin at the **root** of the **trie**.
- **Key to Index Mapping:** Each **key** (e.g., '2', '3', etc.) corresponds to a group of characters. For example, the **key** '2' corresponds to 'd', 'e', and 'f'. These characters have indices 3, 4, and 5 in the **trie**.
- **Recursive Traversal:** For each **key**, we explore all possible branches that correspond to the characters mapped by that **key**. At each level,

we keep track of the path (i.e., the word we're building) and pass this information recursively as we move down the `trie`.

- **Valid Word Check:** When we reach the end of the `key sequence`, we check whether the current `node` represents a valid word. If it does, we add the `word` to the `list` of valid `words`.

Testing with the Kelly-listan (Common Swedish Words)

In this final part of the assignment, we were tasked with populating our T9 `trie` with the `eight thousand` most common words in `Swedish`, provided in the file *kelly.txt*. Once the `trie` was populated, we `encoded` each `word` into a `sequence of keys` and then `decoded` the `sequence` to check if the `word` was correctly identified.

- **Populate the Trie:** Load the *kelly.txt* file into the T9 `trie`.
- **Encode Words:** For each `word`, convert it into a `sequence of T9 keys` using the `character-to-key` mapping.
- **Decode the Sequence:** Use the *decode()* method to verify that the `encoded key sequence` can return the original `word` (or possibly `multiple words`).
- **Test Accuracy:** There may be cases where an `encoded sequence` corresponds to `multiple words`, but most `words` should be `decoded` back to their original form.

For example:

- Word: "svenska" → Encoded: "7836752" → Decoded: "svenska"
- Word: "hej" → Encoded: "435" → Decoded: "hej", "gel", "hel" (multiple possible words)

Results

In the majority of cases, the T9 system was able to correctly `decode` the `key sequences` into valid `Swedish words`. The system worked efficiently, and the `trie` structure allowed for fast `lookups` and `searches`.

Conclusion

In this assignment, we successfully implemented a T9 predictive text system using a **trie data structure**. The **trie** allowed us to efficiently store and retrieve **Swedish words** based on **numeric key sequences**. Although the implementation could be made more memory-efficient, it closely replicates the core functionality of the original T9 **system**.

By handling **encoding, decoding, and managing special characters** like 'å', 'ä', and 'ö', we ensured that the system works seamlessly with **Swedish words**. The testing showed that the system was able to accurately **decode key sequences** into valid words, making this an effective solution for predictive text input.

This assignment reinforced the importance of efficient **data structures** like **tries** and provided valuable experience in handling real-world challenges, such as character **encoding and mapping**.